

EffTE: A Dependency-aware Approach for Test-Driven Ontology Development

Lavdim Halilaj
University of Bonn & Fraunhofer IAIS
Germany
halilaj@cs.uni-bonn.de

Irlán Grangel-González
University of Bonn & Fraunhofer IAIS
Germany
grangel@cs.uni-bonn.de

Steffen Lohmann
Fraunhofer IAIS
Germany
steffen.lohmann@iais.fraunhofer.de

Maria-Esther Vidal
German National Library of Science
and Technology & Fraunhofer IAIS
Germany
maria.vidal@tib.eu

Sören Auer
German National Library of Science
and Technology
Germany
soeren.auer@tib.eu

ABSTRACT

The development of domain-specific ontologies requires joint efforts among different groups of stakeholders, such as ontology engineers and domain experts. By following a *test-driven development* technique, a set of test cases ensures that ontology changes do not violate predefined requirements. However, since the number of test cases can be large and their evaluation time may be high, the ontology development process can be negatively impacted. We propose *EffTE*, an approach for efficient test-driven ontology development relying on a graph-based model of dependencies between test cases. It enables prioritization and selection of test cases to be evaluated. Traversing the dependency graph is realized using breadth-first search along with a mechanism that tracks *tabu* test cases, i.e., test cases to be ignored for further evaluation due to faulty parent test cases. As a result, the number of evaluated test cases is minimized, thus reducing the time required for validating the ontology after each modification. We conducted an empirical evaluation to determine the efficiency of our approach. The evaluation results suggest that our approach is more efficient than an exhaustive evaluation of the test cases; in particular with an increasing ontology size and number of test cases.

CCS CONCEPTS

• **Information systems** → **Resource Description Framework (RDF)**; **Web Ontology Language (OWL)**; Network data models;

KEYWORDS

Ontology Engineering, Test-Driven Ontology Development, Test Cases, Dependency Graph

ACM Reference Format:

Lavdim Halilaj, Irlán Grangel-González, Steffen Lohmann, Maria-Esther Vidal, and Sören Auer. 2018. EffTE: A Dependency-aware Approach for Test-Driven Ontology Development. In *Proceedings of SAC 2018: Symposium on Applied Computing (SAC'18)*. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3167132.3167344>

1 INTRODUCTION

The development of domain-specific ontologies requires joint efforts among different groups of stakeholders, such as ontology engineers and domain experts. Functional requirements can be expressed through *Competency Questions*, which are questions that the underlying ontology should be able to answer [15]. However, the concurrent definition of ontology concepts often results in a violation of the defined requirements, or creates design issues like duplicate entries or missing documentation. For example, in the large and monolithic DBpedia ontology, version 2016-04¹, only 556 from a total of 2,849 properties have an associated label description or a comment [9]. To ensure that any ontology modification has only the expected effects, a set of test cases can be defined based on *Competency Questions*. This is similar to the principles of *test-driven software development*, where test cases (which represent requirements) are defined before the code is actually written [1].

However, since the number of test cases can be large and their evaluation time may be high, the ontology development process can be impacted negatively. Therefore, with a naive approach, test cases are exhaustively evaluated to identify any not intended modification or design issue introduced during the concurrent development of the ontology. On the contrary, the ontology as the main artifact is subject of change due to number of reasons, such as (1) the evolution of the domain, (2) changes in the user perception of the domain, or (3) fixing design issues [10, 12]. Since much effort is invested in creating and extending ontologies for various domains, it is crucial to make ontology development and maintenance cost-effective [8].

In this paper, we propose *EffTE*, an approach for efficient test-driven ontology development. *EffTE* relies on a dependency graph defined by the stakeholders, i.e., ontology engineers, according to their needs; it enables prioritization and selection of test cases to be evaluated. Traversing the dependency graph is performed using *breadth-first* search; *tabu* test cases are tracked and ignored

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SAC'18, April 9–13, 2018, Pau, France

© 2018 Copyright held by the owner/author(s). Publication rights licensed to the Association for Computing Machinery.

ACM ISBN 978-1-4503-5191-1/18/04...\$15.00

<https://doi.org/10.1145/3167132.3167344>

¹<http://wiki.dbpedia.org/services-resources/ontology/>

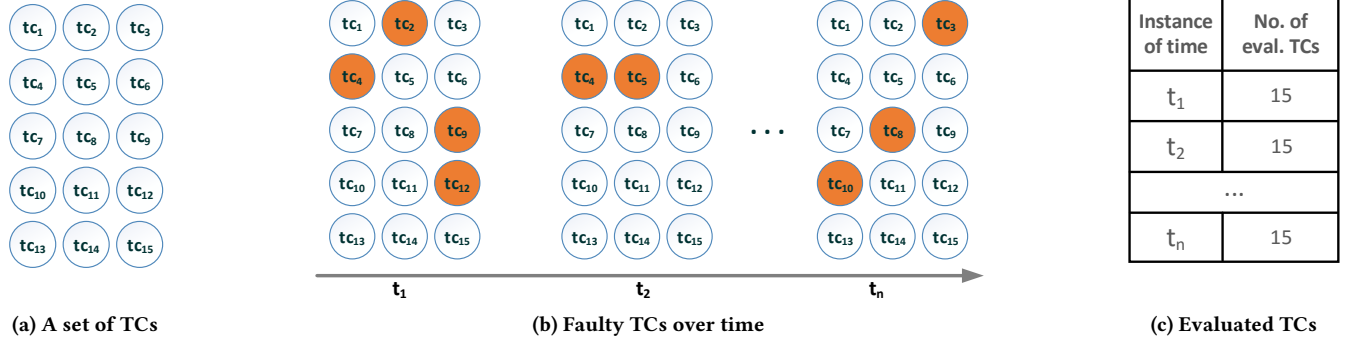


Figure 1: Motivating Example. (a) Test Cases (TCs) to check if an ontology satisfies the design requirements following an entailment regime; (b) faulty test cases (orange) over time after each ontology modification; and (c) number of evaluated test cases per instance of time. For each instance of time, the set of TCs is completely evaluated.

for further evaluation because of faulty parents. As a result, the number of test cases that are evaluated is minimized, thus reducing the time required for ontology validation after each modification. We conducted an empirical evaluation to determine the efficiency of our approach under different conditions, such as changing the number of predefined test cases, ontology sizes, and dependency relationships of test cases. Experimental results suggest that our approach is more efficient than a naive one, in particular with an increasing ontology size and number of test cases.

The remainder of this paper is organized as follows: A motivating example is given in Section 2. Section 3 presents the *EffTE* approach. In Section 4, we outline the implementation of *EffTE*. The approach is evaluated in different scenarios in Section 5, and compared with the state-of-the-art in Section 6. In Section 7, we conclude the paper and give an outlook to possible extensions of this work.

2 MOTIVATING EXAMPLE

Suppose a team composed of ontology engineers and domain experts developing an ontology for a specific domain. After the *requirement gathering* phase, they define questions to be answered by the ontology, e.g., “list all instances of class A that are involved in the activity C”. Further, several guidelines are defined to ensure a certain quality of the ontology. These guidelines include generic constraints that should not be violated during ontology development, e.g., “all concepts must have at least one `rdfs:label` in English” or “all concepts must not share the same English `rdfs:label`”.

As initial step, the ontology engineers define a set of 15 test cases as depicted in Figure 1a. This set ensures that each ontology change is in line with the already defined requirements and guidelines and has only the intended effect based on a given entailment regime². Figure 1b illustrates several faulty test cases after each change of the ontology at different instances of time t_i . For example, in the following time instances t_1 : tc_2 , tc_4 , tc_9 , tc_{12} ; in t_2 : tc_4 , tc_5 ; and t_n : tc_3 , tc_8 , tc_{10} have failed, respectively. Figure 1c shows that all test cases are evaluated at every instance of time t_i , since there is no specified dependency relationship between them. This process is time-consuming, in particular when test cases have a natural

dependency relationship. For instance, checking for duplicate labels should not be performed if *none* of the concepts defined so far do not have labels associated to them; or checking for the existence of subclasses or particular properties should not be performed before the respective classes are defined. Additionally, there are cases where the dependency between test cases is more project-oriented, such as one test case checks for a particular concept or resource; in case of satisfaction, another test case verifies its properties or relations to another concept or resource.

3 EFFTE

In this section, we define the basic terminology and provide a formal description of the *EffTE* approach.

3.1 Problem Statement

Test Suite (TS): A test suite TS represents a set of test requirements $\{r_1, \dots, r_n\}$ that need to be satisfied during the development of an ontology O according to a given *entailment regime*. A test-driven development approach allows for an efficient validation of test cases that comprise a test suite TS . Our test-driven development approach *EffTE* is able to identify the minimal number of test cases required to be evaluated in order to determine if an ontology does not satisfy a test suite TS based on the given *entailment regime*. The following definitions state the core concepts required to formulate the problem tackled by *EffTE*; Table 1 summarizes the notation.

Test Cases (TCs): A test case corresponds to the formal specification of checks required to validate if the requirements in a TS are satisfied in O according to a *entailment regime*. Test cases can be represented using any rule language for ontologies, such as SWRL; however, we focus on test cases expressed as SPARQL queries over concepts defined in O respecting the test suite entailment regime. Given a test suite $TS = \{r_1, \dots, r_n\}$ over an ontology O , an entailment regime ϕ , and a set of test cases STC , the function $\sigma(r_i | STC, O, \phi)$ represents the subset of test cases in STC that need to be evaluated against O to achieve the requirement r_i following the entailment regime ϕ . The evaluation of a test case tc in STC is denoted by $[[tc]]_O^\phi$. The ontology O meets the test case tc following ϕ , iff $[[tc]]_O^\phi$ is different from the empty mapping $\mu_\emptyset$ [11]. Otherwise,

²An entailment regime establishes the entailment relations to be followed in an ontology model.

Table 1: Notation. Symbols used in the test-driven development approach *EffTE*.

Symbols	Description
TS	Test suite with a set of requirements $\{r_1, \dots, r_n\}$
O	Ontology
tc	A Test Case represented as a SPARQL query
ϕ	Entailment Regime
$\sigma(r_i \mid STC, O, \phi)$	Set of test cases in STC associated with a requirement r_i over O using ϕ
$[[tc]]_O^\phi$	Set of mappings resulting of evaluating tc over O following ϕ
$\text{Fault}(TS \mid STC, O, \phi)$	Requirements in TS not achieved in O using ϕ because of faulty test cases in STC
$\mathcal{F}(STC' \mid STC, O, \phi)$	Test cases in $STC - STC'$ that fail as a consequence of failures of test cases in STC'
$\mathcal{T}(STC \mid O, \phi)$	Faulty test cases in STC over O following ϕ
$\mathbb{F}(STC' \mid STC, O, \phi)$	Fault Detection Effectiveness of STC' given O using ϕ and STC ; is a <i>higher is better measure</i> that takes values in $[0.0;1.0]$
$TCG_O^\phi = (STC, E)$	A dependency graph in STC over O using ϕ ; dependencies between test cases are represented in set of edges E

tc is a faulty test case in O w.r.t. the requirement r_i such that $tc \in \sigma(r_i \mid STC, O, \phi)$ is not achieved by O following ϕ .

Faulty Test Case Identification: Given a set STC of test cases implementing a test suite TS , $\text{Fault}(TS \mid STC, O, \phi)$ is a set of requirements in TS associated with at least one test case tc in STC which fails in an ontology O following an entailment regime ϕ .

$$\text{Fault}(TS \mid STC, O, \phi) = \{r_i \mid r_i \in TS \wedge tc \in \sigma(r_i \mid STC, O, \phi) \wedge [[tc]]_O^\phi = \mu_\emptyset\} \quad (1)$$

Determining the faulty test cases from STC requires a considerable amount of time, since all the test cases need to be evaluated over the ontology O according to ϕ . Nevertheless, dependencies between test cases as well as subsumption relationships can be exploited to identify a subset of test cases whose failure implies the failure of the test cases in STC .

Given an ontology O , an entailment regime ϕ , and a set STC' of test cases in STC , the fault detection effectiveness of STC' given O , ϕ , and STC , denoted as $\mathbb{F}(STC' \mid STC, O, \phi)$, corresponds to the fraction of the number of faulty test cases from STC' divided by the number of faulty test cases in STC :

$$\mathbb{F}(STC' \mid STC, O, \phi) = \frac{|\mathcal{F}(STC' \mid STC, O, \phi)|}{|\mathcal{T}(STC \mid O, \phi)| + 1}, \text{ where} \quad (2)$$

- $\mathcal{F}(STC' \mid STC, O, \phi)$ is the set of faulty test cases in STC not included in STC' that fail as a consequence of failures of test cases in STC' (grey nodes in Figure 2b).
- $\mathcal{T}(STC \mid O, \phi)$ is the set of faulty test cases in STC over O following ϕ (orange and grey nodes in Figure 2b).
- $\mathbb{F}(STC' \mid STC, O, \phi)$ is a "*higher is better measure*" that takes values in $[0.0;1.0]$. A value close to 0.0 indicates that a small number of faulty test cases in STC can be detected from STC' , whilst a value close to 1.0 expresses that the number of faulty test cases in STC' and STC are almost the same.

Test Case Prioritization Problem: The *EffTE* approach selects from a set of test cases STC that implement a test suite TS for an ontology O using an entailment regime ϕ , a subset STC' with maximal fault detection effectiveness $\mathbb{F}(STC' \mid STC, O, \phi)$. The problem of

prioritizing test cases in STC over O and ϕ is defined as follows:

$$\begin{aligned} & \underset{STC' \subseteq STC}{\text{argmax}} \mathbb{F}(STC' \mid STC, O, \phi) = \{tc \mid tc \in STC' \wedge \\ & \forall STC'' \subseteq STC, \mathbb{F}(STC'' \mid STC, O, \phi) \leq \mathbb{F}(STC' \mid STC, O, \phi)\} \quad (3) \end{aligned}$$

The *set cover problem* [4] can be reduced to the problem of prioritizing test cases, thus demonstrating the NP-completeness of the test case prioritizing problem.

3.2 Proposed Solution

To solve the problem of prioritizing test cases in STC over an ontology O given an entailment regime ϕ , *EffTE* tracks information about the dependencies between test cases in STC . Furthermore, *EffTE* implements a graph traversal algorithm that explores the test cases STC' whose failure imply the failure of the test cases that depend on them. Thus, the fault detection effectiveness $\mathbb{F}(STC' \mid STC, O, \phi)$ can be maximized.

A Dependency Graph of Test Cases: Given an ontology O following ϕ , a set of test cases STC that implement a test suite TS for the ontology O , a test case dependency graph $TCG_O^\phi = (STC, E)$ is a directed acyclic graph (DAG), where:

- $E \subseteq STC \times STC$ is a set of directed edges representing dependencies between test cases. If $(tc_i, tc_j) \in E$, then the test case tc_j depends on the test case tc_i . Furthermore, tc_j fails whenever the evaluation results of tc_i or tc_j over O following ϕ return the empty mapping $\mu_\emptyset$, i.e., tc_j fails whenever $[[tc_i]]_O^\phi = \mu_\emptyset$ or $[[tc_j]]_O^\phi = \mu_\emptyset$.

The *EffTE* Approach: Figure 2 illustrates the behavior of the *EffTE* approach, contrasting it to Figure 1. *EffTE* relies on modeling the relationships between test cases as a dependency graph $TCG_O^\phi = (STC, E)$ to solve the problem of prioritizing test cases. Thus, *EffTE* identifies a faulty subset STC' of STC , and removes the subadjacent graph from further consideration. The user, i.e., the ontology engineer, can define different TCG_O^ϕ according to a given test suite TS . Figure 2a depicts an exemplary TCG_O^ϕ where tc_5 and tc_9 depend on the outcome of tc_2 . If tc_2 is identified as faulty, such that $[[tc_2]]_O^\phi$ is equal to the *empty mapping* $\mu_\emptyset$, the evaluation of the test cases tc_5 and tc_9 will not be performed. As depicted in Figure 2b, in the time instance t_1 , the faulty test cases are $tc_2, tc_4,$

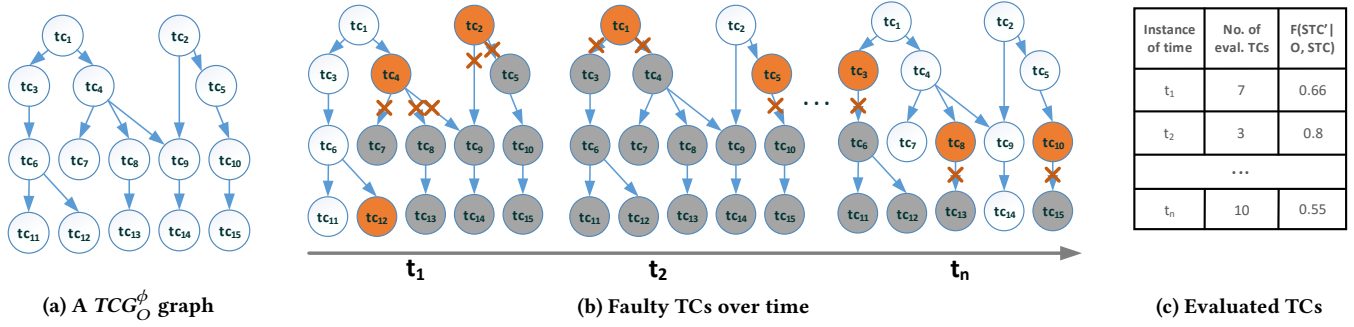


Figure 2: The EffTE Approach. (a) A test case dependency graph TCG_O^ϕ with 15 test cases (TCs); (b) faulty test cases over time (orange nodes are faulty test cases (STC), gray nodes represent TCs ignored for evaluation $F(STC' | STC, O, \phi)$; and (c) number of evaluated TCs and $F(STC' | STC, O, \phi)$ values per instance of time. Dependencies enable to ignore faulty test cases.

Algorithm 1: Breadth-First Search with Tabu mechanism for traversing a TCG_O^ϕ .

```

1 create lists: visitedTCs, nextTCs, tabuTCs, currentTCs;
2 add rootTCs to currentTCs;
3 while currentTCs is not empty do
4   current_tc = currentTCs.dequeue();
5   if (current_tc not in visitedTCs or current_tc.Parents not in
      tabuTCs) and current_tc.Parents are in visitedTCs then
6     if  $[[tc_i]]_O^\phi \neq \mu_0$  then
7       foreach child in current_tc.Children do
8         if child is already defined as parent then
9           STOP the procedure to avoid a cyclic
            possible graph;
10        end
11        add child to nextTCs
12      end
13    else
14      add current_tc to tabuTCs;
15    end
16    add current_tc to visitedTCs;
17  else if current_tc.Parents in tabuTCs then
18    add current_tc to tabuTCs;
19  else if current_tc.Parents not in visitedTCs then
20    add current_tc to pendingTCs;
21  if currentTCs.dequeue() is null AND (nextTCs is not empty
    or pendingTCs is not empty) then
22    add nextTCs and pendingTCs to currentTCs;
23    nextTCs.clear(); pendingTCs.clear();
24  end
25 end

```

and tc_{12} . This enables pruning the graph traversal, and excluding all test cases that depend on the faulty ones from further evaluation. Similarly, in subsequent time instances $t_2, \dots, t_n$, for any faulty tc , the dependent test cases will not be considered for evaluation. As a result, the number of evaluated test cases is reduced. Figure 2c

shows the number of test cases that are evaluated per invocation in the given example, e.g., seven test cases are evaluated in t_1 , only three test cases in t_2 , while in t_n , ten of them are evaluated.

Traversing a Test Case Dependency Graph: We used the *Breadth First Search (BFS)* algorithm for traversing a TCG_O^ϕ to efficiently find the nodes in TCG_O^ϕ that are closer to the root and correspond to faulty test cases. Given a set of root test cases S and their dependencies in E , TCG_O^ϕ is systematically explored by the *BFS* algorithm to evaluate every reachable tc starting from S . As shown in Algorithm 1, after the initial phase of defining S , the procedure continues by recursively iterating on each tc of TCG_O^ϕ . On each iteration, a tc is retrieved from the queue defined as a *FIFO* list and is set as *current_tc*. Three preconditions must be met before the evaluation of *current_tc*: (1) *current_tc* must not have been visited already; (2) all of its parents must have been visited; and (3) none of its parents must have been classified as *tabuTC*. If these preconditions are met, the algorithm continues with the evaluation of *current_tc*.

In case that $[[tc]]_O^\phi$ is equal to the *empty mapping* μ_0 , *current_tc* is added to the list of *tabuTCs* which represents $\mathcal{F}(STC | O, \phi)$. To avoid any possibility of entering in an endless cycle, the children of *current_tc* are checked whether they have been previously classified as *parents* which will cause an immediate stop of the procedure. If not, they are added to the *nextTCs* list to be considered in the next iterations. The iteration over the list of *currentTCs* continues until there is no tc left. Next, all nodes that exist in the lists of *nextTCs* or *pendingTCs* will be added to the list of *currentTCs* for future evaluation. The execution of the algorithm terminates when there is no tc left in *currentTCs*, *nextTCs*, or *pendingTCs*.

4 IMPLEMENTATION

The *EffTE* architecture depicted in Figure 3 consists of three main components: (1) the *Version Control System (VCS)* manages ontology versions via change logs; (2) the *Integrated Validation Service* performs a syntax validation of the ontology and a validation of the ontology against the *STC*; and (3) the *Repository Hosting Platform* stores and propagates ontology changes to other users. *EffTE* is part of VoCol [3], an integrated environment supporting ontology development in distributed settings.

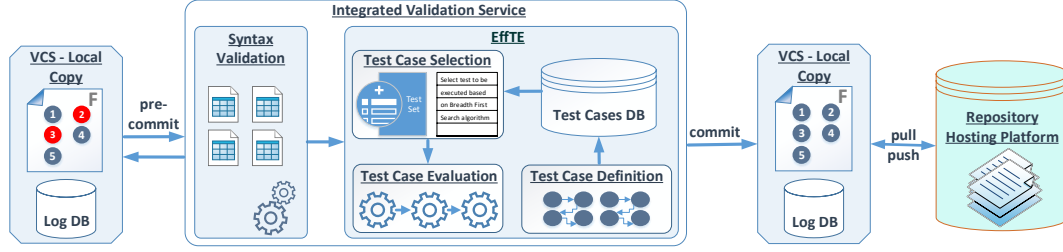


Figure 3: Implementation of *EffTE*. A locally modified ontology file is received, as well as a set of test cases and their dependencies; the result of the validation process is returned as output. An *Integrated Validation Service* validates syntactic errors in an ontology using a *Syntax Validation* component. A *Test Case Validation* component checks the ontology against a set of test cases. The ontology is synchronized with a *Remote Repository* for the distribution of changes.

4.1 Version Control System (VCS)

Managing versions of the ontology files is realized using *Git* as a VCS. The *Git hook* mechanism is utilized to prevent users from committing changes to an ontology file without successfully passing a complete set *STC*. During the committing phase, the *pre-commit* hook is triggered which transfers the modified ontology files to the validation service. The commit is terminated if the modified ontology files are unable to pass the validation process, and the user is notified with an error message comprising the failure details. On the other hand, the commit is successfully completed if no error occurs during the validation process. As a result, a new revision of the modified ontology files is created and the user is able to further proceed with pushing the new version into the remote repository.

Remote hosting platform: *Github* is used as remote hosting platform for saving different versions of the ontology as well as tracking and propagating the changes among users. This facilitates the synchronization of various replicas of the ontology files.

4.2 Integrated Validation Service (IVS)

We implemented *IVS*, an integrated validation service using *NodeJS* as an interoperable cross-platform programming language. It consists of two main components: *Syntax Validation* and *EffTE*. The *IVS* accepts the ontology files as input through an HTTP interface and returns to the client either an error message from the validation process or a successful passing message. The first task after receiving the ontology file is validation for syntactic errors using *Rapper*³. If syntax errors are found, the user receives a report with details about the file name, type, and error line. Next, the *EffTE* component validates the ontology file against a set of test cases. The TCG_O^ϕ is defined by the user as an *Adjacency List* collection and stored in the *Test Cases DB* module. The *Test Case Selection* is responsible for prioritization and selection of the test cases to be evaluated. Each test case is evaluated in the *Test Case Evaluation* module on a given entailment regime where in case of failure, a flag is added to the response message, which prevents users from realizing the commit. Additionally, users receive details regarding the faulty test cases. Otherwise, a correct validation message is provided and the commit is successfully applied. The implementation code of *EffTE* is published on <https://github.com/lavdim/EffTE>.

³<http://librdf.org/raptor/rapper.html>

Table 2: Ontology Description. Different ontology sizes in terms of number of triples, subjects, properties, and objects.

Ontology	# triples	# subjects	# properties	# objects
FOAF	631	86	15	192
Schema.org	8,103	1,569	13	3,545
DBpedia	30,793	3,986	23	16,807

5 EXPERIMENTAL STUDY

We present the results of an experimental study to investigate the efficiency of the *EffTE* approach. The goal of the experiment is to analyze the impact of: (1) ontology size; (2) various TCG_O^ϕ ; and (3) number of test cases on the efficiency of *EffTE*.

We assess the following research questions:

- RQ1)** How does the ontology size impact the efficiency of *EffTE*?
- RQ2)** How do different TCG_O^ϕ affect the efficiency of *EffTE*?
- RQ3)** How does the number of test cases affect the efficiency of *EffTE*?

The experimental configuration is as follows:

Ontologies: We compare the behavior of *EffTE* using three ontologies of different sizes. Table 2 describes these ontologies in terms of number of triples, different subjects, properties, and objects.

- **FOAF**⁴: used to describe persons, activities, and relationships between them and other objects.
- **Schema.org**⁵: describes entities, actions, and their relations to promote the usage of the structured data.
- **DBpedia**: is a cross-domain ontology for describing the information extracted from Wikipedia infoboxes.

```

1 PREFIX rdf: <http://.../22-rdf-syntax-ns#>
2 PREFIX rdfs: <http://.../rdf-schema#>
3
4 SELECT * WHERE { ?s ?p ?o .
5               FILTER (?o = <some-resource> ) }
```

Listing 1: Simple SPARQL SELECT query, where <some-resource> is replaced to generate a faulty test case.

⁴<http://xmlns.com/foaf/spec/>

⁵<http://schema.org>

Table 3: Experimental Set-Up. Three scenarios varying: Ontology sizes, dependency graph topology, and number of test cases (TCs). Different faulty TCs are generated for first four instances of time (TI), whereas in TI-5, zero faulty TCs are enforced.

Scen.	Ontology	No. of TCs	Topology of TCG_O^ϕ	Faulty TCs per TI				
				TI-1	TI-2	TI-3	TI-4	TI-5
1	Foaf	10	(1,{3,8}),(2,{6}),(3,{5}), (4,{6,8,10}),(5,{7,9}),(6), (7,{10}),(8,{9}),(9), (10)	3, 4, 7	3	1, 5, 7, 9	2, 7, 5, 8	-
	Schema.org							
	DBpedia							
2	DBpedia	10	(1,{9,10}),(2,{4,6}),(3,{5}),(4,{8}),(5),(6,{7}),(7),(8),(9,{10}),(10)	3, 4, 7	3	1, 5, 7, 9	2, 7, 5, 8	-
			(1,{3,8}),(2,{6}),(3,{5}),(4,{6,8,10}),(5,{7,9}),(6),(7,{10}),(8,{9}),(9),(10)					
			(1,{5,9}),(2,{3,7}),(3,{5,7,8}),(4,{5}),(5),(6),(7,{9,10}),(8),(9),(10)					
3	DBpedia	10	(1,{3,8}),(2,{6}),(3,{5}), (4,{6,8,10}),(5,{7,9}),(6), (7,{10}),(8,{9}),(9),(10)	3, 4, 7	3	1, 5, 7, 9	2, 7, 5, 8	-
		20	(1),(2,{9,12,15}),(3,{8,13,18}),(4,{8,16,19}), (5,{13,17}),(6,{9,18}),(7,{8,13,16}),(8,{13,16}), (9,{17}),(10),(11,{15,17}),(12),(13,{18}),(14),(15,{20}),(16), (17,{19}),(18,{19}),(19),(20)	1, 6, 12, 16, 19	2, 4, 8, 9	2, 3, 9, 14, 20	1, 4, 19	-
		30	(1,{2}),(2,{19}),(3,{6,10}),(4,{15,22,30}), (5,{23,30}),(6,{9,13}),(7,{15,17}),(8,{13,15}),(9,{12,26}),(10,{25}),(11,{12,24}),(12),(13,{20,24,26}),(14),(15,{18,21,7}),(16,{24}),(17,{18}),(18),(19,{26}),(20),(21),(22,{24}),(23,{28}),(24,{27}),(25,{28}),(26,{28,30}),(27,{29}),(28,{29}),(29),(30)	3, 18	7, 9, 13, 22, 28	6, 7, 10, 14, 23	2, 8, 18	-

Test Case Definition: With the objective of achieving a fair comparison, and avoiding any impact on the evaluation time that arises from the complexity of the query itself, we use the query depicted in Listing 1. It represents a simple *SPARQL SELECT* query where its failure is simulated by replacing the *<some-resource>* variable with a particular concept or instance of some concept. The *RDFS* entailment regime is used in all the test cases.

Dependency Graph Generation: Different dependency graphs TCG_O^ϕ of the test cases are randomly generated using the *randomDAG* function of the *RStudio* platform. The parameter *prop* denotes the probability of connecting a test case to another test case with higher topological ordering, e.g., *prop* = 0.2 generates TCG_O^ϕ where test cases have an average of two parent test cases.

Test Case Failure Generation: We randomly generate a number of faulty test cases following a *Poisson distribution*, i.e., test cases that fail after each change realized by users, presuming that these failures occur according to a *Poisson distribution*. The parameter λ indicates the average number of faulty test cases after each ontology change, i.e., $\lambda = 2$ simulates that on average two faulty test cases occur after each change. Following a *Uniform distribution* with replacement, various test cases are randomly chosen to fail.

Metrics: We report on the following metrics: (1) Evaluation time (**ET**), which corresponds to the time required for evaluation of a *STC*; and (2) number of evaluated test cases (**ETC**).

Baseline: The baseline represents: (1) time required for evaluation of a *STC* by a naive approach; and (2) the maximum number of evaluated test cases, which corresponds to the number of test cases evaluated in a naive approach. The naive approach corresponds to an exhaustive evaluation of all test cases, where neither order nor dependency information is considered.

Implementation: Experiments are run on a Linux Ubuntu 16.04 machine, with a 6th Gen Intel Core i7-6820HQ, 2.70GHz, and 16GB RAM 2133MHz DDR4. *EffTE* is implemented using Node.js version 4.4.5. The *Syntax Validation* is implemented using Rapper version

2.0.15, whereas the *Test Case Validation* component uses the Java Apache Jena libraries version 3.0.1. Git version 2.7.4 is used as the VCS. Generation of the faulty test cases for different scenarios is implemented using functions for *Poisson Distribution* and *Uniform Distribution* in *RStudio* version 1.0.136.

Method: Three different scenarios are defined to answer the above mentioned research questions by changing: 1) the size of the ontology; 2) the topology of TCG_O^ϕ ; and 3) the number of test cases. On each scenario, experiments are run twice in each time instance (*TI*), using the naive and *EffTE* approach. In time instances *TI-1:4*, different test cases are randomly chosen to fail following a *Uniform distribution* with replacement. We enforce a *zero-faulty tc* in *TI-5*, to compare the efficiency of the naive approach and *EffTE* under similar conditions. Table 3 lists the configuration of each scenario. A dependency graph TCG_O^ϕ is defined as follows: for instance, (1, {3, 8}) represents test cases *tc_3* and *tc_8* that depend on *tc_1*. The graphical depictions of various TCG_O^ϕ topologies which comprise different numbers of test cases can be found on <https://figshare.com/s/0d61829a1df65e69fda1>.

5.1 Impact of the Ontology Size

To answer **RQ1**, we follow the above described method for assessing the efficiency of the naive approach and *EffTE* with three different ontologies (cf. Table 2). Figure 4a shows the evaluation time for a *STC* required by the naive approach and *EffTE*. In the three ontologies, the *ET* values are lower for *EffTE* compared to the naive approach in time instances *TI-1,2,3,4*. We observe that the *ET* values of *EffTE* and the naive approach are only slightly different in case of the *FOAF* ontology. The reason is that the *FOAF* is a small-size ontology and query execution time is relatively low. In the case of *Schema.org*, which is a medium-size ontology, an increased difference of *ET* values between two approaches is recognized. Finally, *EffTE* requires lower *ET* values in the first four instance of time compared to the naive approach for the *DBpedia Ontology* which consists of more than 30K triples.

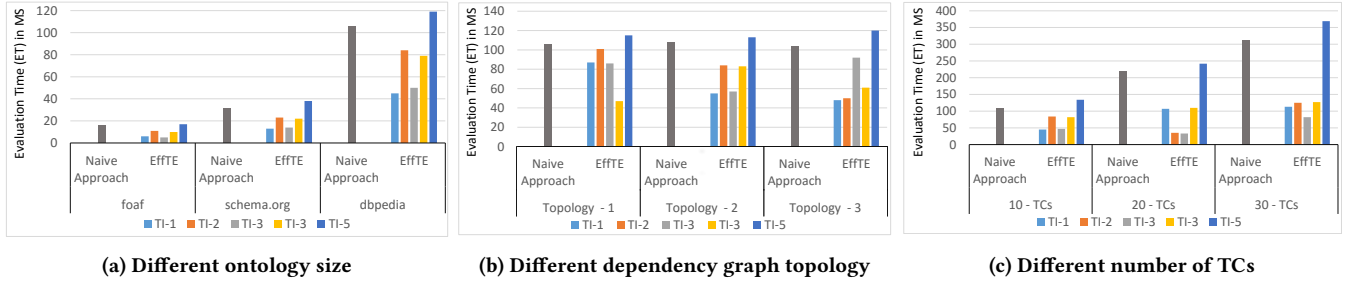


Figure 4: Evaluation Time (ET): Naive Approach vs. *EffTE*. Impact of different scenarios on Execution Time (ET): (a) Ontology size; (b) dependency graph topology; and (c) and number of test cases. Results suggest that *EffTE* exhibits better ET values in four instances of time *TI-1,2,3,4* whereas in *TI-5* it performs worse, since there is no faulty test case



Figure 5: Number of Evaluated Test Cases (ETC): Naive Approach vs. *EffTE*. Impact of different scenarios on ETC: (a) Dependency graph topology; and (b) number of test cases. Results suggest that *EffTE* exhibits better ETC values in four instances of time *TI-1,2,3,4* whereas in *TI-5* it has same ETC values as a naive approach, since there is no faulty test case

5.2 Impact of the Topology of TCG_O^ϕ

With the aim of answering research question **RQ2**, various TCG_O^ϕ are defined, i.e., test cases have different dependencies between each other. As shown in Table 3, *Scenario 2*, three different TCG_O^ϕ with 10 test cases each are generated. For every TCG_O^ϕ , a particular number of test cases are chosen to fail. The results in Figure 4b show that *EffTE* exhibits better performance on time needed to evaluate all the dependency graphs on four instances of time. *EffTE* will require additional time in *TI-5* because of no faulty *tc*. As shown in Figure 5a, the ETC values are less in *EffTE* compared to the naive approach in the first four instances of time.

5.3 Impact of the Number of the Test Cases

We define *Scenario 3* (cf. Table 3), to answer **RQ3**. This scenario comprises three TCG_O^ϕ composed of 10, 20, and 30 test cases, respectively. Similar to other scenarios, for each TCG_O^ϕ , a number of test cases are chosen to fail. From the Figure 4c, we can observe that with the increase number of test cases, *EffTE* exhibits better performance on the time needed for the evaluation of a TCG_O^ϕ in time instances *TI-1,2,3,4*. As expected, *EffTE* requires a bit more time in *TI-5* than the naive approach in case of absence of faulty test cases. Additionally, Figure 5b depicts the ETC values of two approaches for different number of predefined test cases. We note that ETC values are considerably lower in *EffTE*, in first four instances of time where faulty test cases exists.

5.4 Discussion

As we can observe, in the first four instances of time of each scenario, *EffTE* is performing better than the naive approach for a given set of test cases *STC* using an entailment regime ϕ . However, as shown in *TI-5*, *EffTE* may also need more time for the evaluation process, i.e., ET values are higher in *EffTE*. This behavior occurs when there are no faulty test cases and additional workload is only used for traversing a dependency graph TCG_O^ϕ . Although the test case (cf. Listing 1) to be evaluated is intentionally chosen to be the "simplest" one, the reported time from the experiments shows an improvement of hundreds of milliseconds. This improvement becomes more evident in cases when the size of the ontology is not small and increases progressively in time. Moreover, if the complexity of the test case increases, i.e., *checking for duplicate labels* particularly in ontologies with a large number of concepts, avoiding such kind of test cases from further evaluation if *none of the defined concepts do not have label associated* becomes a crucial aspect in the development process. On the other hand, the Figure 5 shows that *EffTE* reports lower ETC values, i.e., a lower number of evaluated test cases in time instances *TI-1,2,3,4*, whereas in *TI-5*, it shares the same ETC values with the naive approach. As a conclusion, whenever exists a faulty test case that has other test cases depending on its outcome, the overall number of evaluated test cases is considerably lower using the *EffTE* approach. In the worst case, the number of evaluated test cases in *EffTE* is equal with the number of evaluated test cases in a naive approach.

6 RELATED WORK

Many research approaches have been proposed to support ontology engineers in developing ontologies according to domain requirements while at the same time avoiding constraints violation. In this regard, Ren et al. [13] present an approach for enabling ontology engineers to formulate machine processable *Competency Questions* (CQs). The implemented solution allows users to utilize the predefined CQs or to enter new CQs in a controlled natural language. As a result, appropriate *Authoring Tests* are generated and automatically executed to check whether an ontology meets predefined requirements. An approach for test-driven development of ontologies is described in [5]. It provides a number of predefined tests which help ontology developers to verify which axiom is missing and how to properly add it. Unit Tests for Ontologies [16] is transfers principles of software engineering techniques to test-based development of ontologies. Two ontologies, *positive test ontology* T^+ and *negative test ontology* T^- are created to force the developed ontology to follow constraints defined in T^+ and avoid constraints defined in T^- . SHACL[6] is a language for validating RDF graphs against a set of conditions provided as *shapes* and other constructs expressed in the form of an RDF graph. A methodology for assessing the quality of Linked Data is described in [7]. It uses the concepts of test-driven software development such as test cases to ensure a basic level of quality. Test cases defined as SPARQL templates later are materialized as concrete test queries for assessing the quality of the ontologies and knowledge bases. OntologyTest [2] is a tool that supports specification of different types of tests such as *instantiation tests*, *recovering tests*, or *satisfaction tests*. These tests check for the predefined ontology requirements, and can be run all at once, by grouping them into categories or individually.

In software development, *regression testing* is the process of retesting the software with a test suite after each modification. However, due to large amount of time needed for *regression testing*, much research has been conducted in areas such as: *minimization*, by eliminating redundant test cases; *selection*, by identifying relevant test cases based on recent changes; and *prioritization*, by ordering the test cases based on specific criteria [17]. Various techniques are developed in this regard. Particularly relevant for us are studies focused on so-called *Graph-Walk Approaches* such as Control Dependency Graphs (CDGs). The CDGs utilizes a depth-first algorithm to traverse the graph for selecting a subset of test cases according to the modified code [14].

In contrast to the aforementioned works, *EffTE* focuses on an ontology as the main artifact along with test-driven development technique, where a development process is realized following the principle of first defining the test and later writing the code that passes that test. Moreover, *EffTE* utilizes a BFS algorithm for traversing a TCG_O^ϕ . An additional mechanism, called *Tabu*, stores faulty test cases and enables pruning the graph, thus avoiding subsequent test cases from further evaluation.

7 CONCLUSION AND FUTURE WORK

This paper presented *EffTE*, an approach for efficient test-driven development of ontologies. *EffTE* relies on a dependency graph TCG_O^ϕ modeled by users according to their requirements on a given entailment regime; dependency graphs enable prioritization of the

test cases. Traversing a TCG_O^ϕ is realized by a BFS algorithm along with an additional mechanism that stores *tabu* test cases, i.e., test cases to be ignored for further evaluation because of faulty parents. As a result, the number of evaluated test cases is minimized, thus the fault detection effectiveness $\mathbb{F}(STC' \mid STC, O, \phi)$ is increased and the test case validation time is reduced. We performed an empirical evaluation to study the efficiency of *EffTE* compared to a naive approach on various scenarios with different ontology size, dependency graph topologies, and number of test cases. The results suggest that in all cases where a faulty test case exists, the efficiency of *EffTE* is higher compared to a naive approach.

In future work, we envision to improve the approach with mechanisms for dynamic prioritization and selection of the test cases and limit their evaluation only to the recent ontology modifications. Additionally, parameters will be considered such as users who perform changes and modified ontology files (if an ontology is developed in multiple files). Thus, a user- and file- based selection of test cases can be achieved; it will enhance the ontology development process on collaborative environments.

8 ACKNOWLEDGMENTS

This work has been supported by the European Union's Horizon 2020 programme in the context of the projects BigDataEurope (grant no. 644564) and Biotope (grant no. 688717).

REFERENCES

- [1] S. Fraser, K. Beck, B. Caputo, T. Mackinnon, J. Newkirk, and C. Poole. Test driven development (tdd). In *4th Int. Conf. on Extreme Programming and Agile Processes in Soft. Eng.*, XP'03, pages 459–462, Berlin, Heidelberg, 2003. Springer-Verlag.
- [2] S. García-Ramos, A. Otero, and M. Fernández-López. Ontologytest: A tool to evaluate ontologies through tests defined by the user. In *IWANN 2009 Workshops*, pages 91–98, 2009.
- [3] L. Halilaj, N. Petersen, I. Grangel-González, C. Lange, S. Auer, G. Coskun, and S. Lohmann. Vocol: An integrated environment to support version-controlled vocabulary development. In *Knowledge Engineering and Knowledge Management - 20th Int. Conf., EKAW, Bologna, Italy, Proceedings*, pages 303–319, 2016.
- [4] R. M. Karp. Reducibility among combinatorial problems. In *Symposium on the Complexity of Computer Computations, March 20-22, 1972, IBM Thomas J. Watson Research Center*, pages 85–103, 1972.
- [5] C. M. Keet and A. Lawrynowicz. Test-driven development of ontologies. In *13th Extended Semantic Web Conf. ESWC*, pages 642–657, 2016.
- [6] H. Knublauch and D. Kontokostas. Shapes constraint language SHACL, 2017. Accessed: 2017-05-08.
- [7] D. Kontokostas, P. Westphal, S. Auer, S. Hellmann, J. Lehmann, R. Cornelissen, and A. Zaveri. Test-driven evaluation of linked data quality. In *23rd Int. World Wide Web Conf. WWW*, pages 747–758, 2014.
- [8] M. Mehrotra. Ontology analysis for the semantic web. In *Ontologies and the Semantic Web: AAAI Workshop, Technical Report WS-02-11*. AAAI Press, 2002.
- [9] N. Mihindukulasooriya, M. Poveda-Villalón, R. García-Castro, and A. Gómez-Pérez. Collaborative Ontology Evolution and Data Quality - An Empirical Analysis. In *13th OWL: Experiences and Directions Workshop*, 2016.
- [10] N. F. Noy and M. C. A. Klein. Ontology evolution: Not the same as schema evolution. *Journal Knowledge and Information Systems*, 6(4):428–440, 2004.
- [11] J. Pérez, M. Arenas, and C. Gutierrez. Semantics and complexity of sparql. *ACM Trans. Database Syst.*, 34(3):16:1–16:45, Sept. 2009.
- [12] P. Plessers and O. D. Troyer. Ontology change detection using a version log. In *4th Int. Semantic Web Conf. (ISWC)*, pages 578–592, 2005.
- [13] Y. Ren, A. Parvizi, C. Mellish, J. Z. Pan, K. van Deemter, and R. Stevens. Towards competency question-driven ontology authoring. In *11th Extended Semantic Web Conf. (ESWC)*, pages 752–767, 2014.
- [14] G. Rothermel and M. J. Harrold. A safe, efficient algorithm for regression test selection. In *Conf. on Soft. Maintenance (ICSM)*, pages 358–367, 1993.
- [15] M. Uschold and M. Gruninger. Ontologies: principles, methods and applications. *Knowledge Eng. Review*, 11(2):93–136, 1996.
- [16] D. Vrandečić and A. Gangemi. Unit tests for ontologies. In *On the Move to Meaningful Internet Systems, OTM Workshops and Posters*, pages 1012–1020, 2006.
- [17] S. Yoo and M. Harman. Regression testing minimization, selection and prioritization: a survey. *Softw. Test., Verif. Reliab.*, 22(2):67–120, 2012.